Практическая работа 8. Проектирование и программирование пользовательского интерфейса

Промышленные приложения неизменно являются **клиент/серверными** (client/server — C/S) решениями. Часто они являются **многоярусными** решениями. Ярусы определяют ряд уровней серверов, типа Web-сервера, сервера приложения и сервера БД. Клиентом может быть рабочий стол, портативный компьютер, карманный компьютер, мобильный телефон, терминал ввода/вывода и т. д. Приложение может поддерживать многих клиентов.

Один из способов классификации клиентов состоит в разделении их на программируемых клиентов и клиентов-браузеров [94]. **Программируемый клиент** предполагает, что программа находится и выполняется на клиенте и имеет доступ к ресурсам, хранящимся на клиенте. Также важно, что программируемый клиент может загружать данные с сервера, отключаться от источника данных, если это потребуется, локально кэшировать данные, обрабатывать их и передавать их своему пользовательскому интерфейсу. Программируемый клиент иногда называется **толстым клиентом** или **богатым клиентом**.

Клиент-браузер нуждается в сервере, чтобы загрузить требуемые данные и получить инструкции для предоставления данных в пользовательском интерфейсе на основе Web-технологии. Кроме простых проверок пользова-

тельского входа клиент-браузер не имеет возможности своей собственной обработки. Данные представляются как Web-страницы, отформатированные на языке разметки гипертекстов (HyperText Markup Language — HTML). Клиент-браузер также называется **тонким клиентом**, **Web-клиентом** или **HTML-клиентом**.

Приложение типа учебного примера ЕМ может быть развернуто:

- *локально* с программируемым клиентом (клиент Java в случае ЕМ);
- на **Web-сервере** с клиентом-браузером и обращением с помощью сервлетов и серверных страниц на Java (Java Server Pages — JSP);
- на **сервере приложения** типа сервера Enterprise JavaBeans (EJB) с программируемым клиентом или клиентом-браузером.

Учебный пример ЕМ предполагает, что программируемый клиент Java развернут локально. Однако чтобы объяснить клиенты-браузеры и различные опции развертывания, части итераций 2 и 3 ЕМ-решения были перепрограммированы. В этой главе обсуждается проектирование и программирование пользовательского интерфейса для клиента Java-приложения, который использует набор базовых классов Java (Java Foundation Classes — JFC) с библиотекой Swing API. Она также представляет пользовательский интерфейс для приложения, разработанного с использованием бизнес-компонент, и такой, у которого бизнес-логика «вытекает» из исходного проекта БД. Такое приложение обычно разворачивается на сервере приложения.

8.1. Основные принципы проектирования пользовательского интерфейса

Интерфейс — термин в разработке систем, который может иметь разный смысл. В объектной технологии интерфейс — объявление множества особенностей, которые непосредственно не инстанциируемы[1]. **Предоставленный интерфейс** класса — множество интерфейсов, реализованных классом. Множество интерфейсов, требуемых классом от других классов, обычно называется **требуемым интерфейсом**. Множество общедоступных методов в классе называется его **общедоступным интерфейсом** (или протоколом). **Проектирование интерфейса** означает также все, что должно установить связь между компонентами программного/аппаратного обеспечения. Ясно, что понятие *пользовательского интерфейса* не связано с другими понятиями интерфейса.

Жизненный цикл проектирования пользовательского интерфейса — неотъемлемая часть жизненного цикла разработки системы. Ушли те дни, когда проектирование пользовательского интерфейса было последним в проектировании системы/программы. Они ушли с исчезновением принципа процедурного программирования, воплощенного системами КОБОЛа. Современные системы переключили этот принцип с «управления программой» к «управлению пользователем», то есть пользователь решает, *что* программа делает, *какие* действия она исполняет и *когда* эти действия выполняются.

[1] Инстанциируемый структурный тип — тип, обладающий конструктором для создания объекта, не инстанциируемый структурный тип — тип, не обладающий конструктором. (*Прим. перев.*)

Соответственно, проектирование пользовательского интерфейса переместилось от преимущественно задач проектирования/реализации системы к операциям, начинающимся на самых ранних стадиях анализа требований. В современной практике, которой следует эта книга, эскизы окон пользовательского интерфейса часто включаются в документы сценария использования. Эти ранние эскизы чрезвычайно полезны для документа-ции и проверки функциональных требований пользователя.

Включение эскизов пользовательского интерфейса в документы сценария использования помогает в определении функциональных требований разрабатываемой системы. Это — только исходный и дополнительный аспект проектирования пользовательского интерфейса. **Проект пользовательского интерфейса** обеспечивает пользователю доступ к функциональным возможностям системы, но как автономная операция, он в первую очередь концентрируется на **применимости** системы. Применимость — одна из особенностей FURPS+

Определение применимости включает впечатление и ощущение от использования пользовательского интерфейса, но его главные характеристики — простота использования системы, простота изучения, обеспечение пользовательским интерфейсом эффективности работы пользователя, скорость действий, сопротивляемость к ошибкам и восстанавливаемость после них, адаптируемость среды пользовательского интерфейса. Короче говоря, рискуя некоторой неточностью, можно сказать, что удобная система — это дружественная система.

Проект пользовательского интерфейса должен следовать ряду важных директив (принципов), чтобы тот считался «хорошим проектом». Директивы стоят выше детальных технических соображений. А применимость находится в центре этих руководящих требований.

Исторически, наиболее технически обоснованные директивы проектирования пользовательского интерфейса были представлены в контексте конкретных вида и поведения. Цель состояла в том, чтобы показать, как директива могла бы быть реализована в виде и поведении Win32 (Windows), Macintosh, Motif (Unix) и т. д. В современной практике использования нескольких платформ и настраиваемых во время выполнения вида и поведения могут быть представлены директивы сами по себе, а соответствующие вид и поведение могут быть выбраны позже.

Директивы проектирования пользовательского интерфейса можно объяснять с различными степенями детализации и совмещения. В дан-ной книге они сгруппированы в следующие категории:

- пользователь в управлении;
- непротиворечивость интерфейса;
- снисходительность интерфейса;
- адаптируемость интерфейса.

8.1.1. Пользователь в управлении

Директива «Пользователь в управлении» подобна предписанию, которое существенно возвышается над всеми другими директивами. Эта директива

охватывает характер взаимодействий между пользователем и программой. При современных графических и основанных на Web-технологии пользовательских интерфейсах пользователь инструктирует программу относительно каждого следующего шага, который необходимо выполнить. Даже в то время, когда программа занята выполнением некоторой обработки, а пользователь ожидает получение результата, пользователь должен сохранять ощущение управления.

Точности ради следует сказать, что директива «Пользователь в управлении» на самом деле лишь обеспечивает ощущение управления пользователем. Это иногда называется *непорождающим* принципом. Пользователь не должен иметь чувства, что требуется постоянно узнавать у программы, что делать. В любое время пользователь должен иметь возможность инструктировать программу, что делать — выбирать следующий шаг, изменять направление действия, прерывать текущую задачу и т. д.

Директиве «Пользователь в управлении» непосредственно помогает объектная технология. Фактически, современная популярность объектной технологии может быть приписана тому совпадению, что она в 1980-х годах была обеспечена появляющимися графическими пользовательскими интерфейсами. Объектно-ориентированная программа состоит из большого количества сотрудничающих объектов. Каждый объект — маленький программный модуль с набором услуг, которые могут быть выполнены, когда запрашиваются другими объектами или когда активизируются непосредственным событием, поступающим от пользователя. За каждым взаимодействием пользователя с программой стоит объект, который понимает запрос пользователя.

Чтобы пользователь чувствовал управление, программа должна быть соответствующим образом разработана. Пользовательский интерфейс должен быть удобен. Он должен использовать терминологию бизнес-области, а не компьютерный жаргон. Это должно минимизировать изменения интерфейса в результате изменений в состоянии системы. Пользователь не должен постоянно удивляться непригодностью пунктов меню, командных кнопок и других средств управления в ситуациях, когда это средство управления было бы доступно при выполнении подобных задач в подобных окнах. Любое руководство, предоставляемое программой пользователю, должно быть сдержанно и корректно.

Директива «Пользователь в управлении» предполагает надлежащую и своевременную обратную связь от программы, когда та выполняет задачу, которая не может быть прервана. Это — преимущественно проблема выполнения функций. Пользователь начинает беспокоиться, если программа выполняет некоторую обработку более долго, чем обычно, и пользователь не информирован относительно того, что происходит. Могут иметься серьезные причины, почему программа работает медленно. Это может быть из-за необычной рабочей нагрузки или из-за потребности выполнить системой административную задачу, типа реорганизации индексов в реляционной БД. Ни один из этих факторов не освобождает проектировщика пользовательского интерфейса от необходимости обеспечить пользователя своевременной информационной обратной связью.

8.1.2. Непротиворечивость интерфейса

Непротиворечивость при проектировании интерфейса означает соответствие промышленным стандартам и стандартам организации в отношении именования, кодирования, сокращений, размещения визуальных объектов, использования средств управления, включая меню, кнопки, функции клавиатуры, и т. д. Непротиворечивость интерфейса не является особенностью отдельной изолированной прикладной программы. Непротиворечивость интерфейса относится ко многим приложениям в пределах системы предприятия и/или ряда приложений, выполненных на одной и той же платформе создания пользовательского интерфейса.

Индивидуальные творческие убеждения, склонности к изобретательности, отходы от стандартов ради создания более привлекательных вещей и т. д. — не приветствуются при проектировании пользовательского интерфейса. Они только смущают пользователей и отрицательно воздействуют на директиву «Пользователь в управлении». Стандарты могут быть не лучшим отражением состояния мастерства, но проектировщик должен помнить, что практически единственная вещь, относительно которой должен заботиться пользователь, состоит в том, чтобы сделать работу самым простым, самым быстрым и наиболее знакомым путем.

Однако непротиворечивость не означает повторения снова и снова одних и тех же ошибок. Имеются пределы непротиворечивости и стандартов. Стандарты имеют тенденцию узаконить и хорошую, и плохую практику и препятствовать прогрессу. Усовершенствования ПО системы и аппаратных средств ЭВМ, новые вычислительные способности, создание приложений и даже социальные, политические, юридические или экономические факторы могут потребовать новшеств и несоблюдения стандартов в новых приложениях.

Непротиворечивость интерфейса подытоживает эстетику интерфейса. Можно предположить, что непротиворечивый интерфейс, соответствующий стандартам, приведет к эстетическому интерфейсу — интерфейсу, привлекательному визуально, с ощущением баланса и симметрии, с хорошим использованием цвета, привлекательным выравниванием и интервалом между элементами, приятной группировкой связанных элементов, без специальных требований к перемещению взгляда человека и т. д.

8.1.3. Снисходительность интерфейса

Приложение должно быть снисходительным к своим пользователям, выполняющим недопустимые действия, активизирующим недопустимые события, вводящим ошибочные данные и т. д. **Снисходительный интерфейс** подразумевает эластичный интерфейс — интерфейс, который может изящно приспосабливаться к исключениям и ошибкам. Снисходительный интерфейс подразумевает интерфейс, с которым легко экспериментировать, — интерфейс, который поощряет пользователя экспериментировать с неизвестными опциями и который обеспечивает принцип «Пользователь в управлении».

Снисходительный интерфейс обеспечивает надлежащую обратную связь пользователю и дружественную обработку исключений. Точка зрения программы на пользователя должна быть такой, как точка зрения продавца на

клиента: она не зависит от того, прав ли клиент. Пользователь может делать любую ошибку в суждении, выполнять любое действие, активизировать любую команду меню, нажимать любую командную кнопку, и при всем этом программа не должна нарушить свою работу. Глупое действие не должно выполняться. Недопустимое действие должно быть обработано.

Невыполнение действия пользователя означает, прежде всего, простую задачу забыть то, что делал пользователь. Это действительно имеет место в однопользовательском автономном приложении типа Microsoft Word. Опция «Отменить ввод» в Microsoft Word позволяет удалить не только самое последнее, но и большое количество последних изменений в документе Word. Не так обстоит дело в случае типичного промышленного приложения, обращающегося к БД и выполняющего бизнес-транзакции к сохраняемому размещению БД. Выполненная транзакция не может быть уничтожена. В лучшем случае она может быть компенсирована.

Как скомпенсировать транзакцию? Предположим, что пользователь забрал деньги со счета банка. Ясно, что, будучи совершенной, эта транзакция не может быть уничтожена. Но ей можно дать компенсацию. Пользователь должен иметь возможность поместить деньги назад на счет. Компенсирующая транзакция не является автоматической операцией системы. Пользователь должен вызвать ее. Но даже и тогда компенсирующая транзакция может потерпеть неудачу.

Представьте обратную банковскую транзакцию, в которой пользователь внес деньги в объединенный счет. Представьте затем, что пользователь решает «удалить» транзакцию вскоре после этого и забрать депонированные деньги. Транзакция изъятия может и не состояться, если тем временем другой держатель счета обратился к счету с другого места и забрал деньги, только что депонированные.

Снисхождение включает реакцию пользовательского интерфейса на недопустимое действие пользователя. Во-первых, недопустимое действие не должно нарушить работу системы. Во-вторых, недопустимое действие нельзя разрешить. В-третьих, недопустимое действие должно быть обработано мягко. Пользователь должен быть информирован относительно характера проблемы, должен быть «просвещен», почему система не может выполнить действие, и должно быть сказано, как исправить проблему и какое альтернативное действие может быть позволено.

7.1.4. Адаптируемость интерфейса

Пользовательский интерфейс должен иметь возможность приспособить любого пользователя. Разнообразие пользователей имеет множество измерений. Во-первых, пользователь может быть пользователем-новичком или опытным пользователем. Во-вторых, пользователи могут прибывать с различных мест действий. В-третьих, пользователь может быть человеком-инвалидом.

Проект пользовательского интерфейса должен быть **адаптирован** так, чтобы его особенности могли быть изменены в зависимости от знания его пользователем. Знание имеет два аспекта. Первый аспект касается знания пользовательского интерфейса как механизма взаимодействия с системой и способности «перемещаться» и «продвигаться». Второй аспект касается зна-

ния, как «получить нужное» в смысле бизнеса. Этот аспект предполагает пользователя с хорошим знанием бизнес-области приложения.

Хороший пользовательский интерфейс должен обеспечить настройку и персонализацию средства обслуживания, чтобы позволить пользователю изменять особенности пользовательского интерфейса в зависимости от уровня компьютерных и/или бизнес-знаний. Настройка — операция объявления программе уровня знаний о ней пользователя, чтобы пользовательский интерфейс мог бы изменить ее внешний вид и поведение. Персонализация имеет меньше дела со знаниями пользователя и больше с предпочтениями пользователя выполнять операции.

Проектирование пользовательского интерфейса должно адаптироваться так, чтобы его особенности могли быть изменены в зависимости от места действия пользователя. Информация о месте действия — особенность современных Интернет-систем. Приложение может запросить операционную систему, на которой оно выполняется, чтобы получить информацию о месте действия, и затем может быть приспособлено к этому месту действия. Адаптация включает изменение языка пользователя (например, с английского на итальянский), набора символов и шрифтов, валюты, времени, порядка сортировки, формата даты, форматов для почтовых адресов и номеров телефона, способа, как называются люди и как на них ссылаются, пиктограмм и модельных представлений (типа значения цвета в различных культурах) и т. д.

Проектирование пользовательского интерфейса должно быть адаптируемо так, чтобы инвалиды могли использовать его средства. Различные нарушения требуют различных мер адаптируемости. Слепые люди требуют, чтобы приложение использовало систему Брайля или речь. Глухие люди требуют визуальной замены для любой звуковой продукции. Физически ущербные люди могут потребовать, чтобы в систему была включена способность распознавания голоса. Адаптированный пользовательский интерфейс должен поддерживать альтернативные устройства ввода и вывода типа терминалов Брайля, считывателя экрана, луп экрана и подобных устройств.

8.2. Компоненты пользовательского интерфейса

Программирование пользовательского интерфейса пользуется преимуществом широко доступных **библиотек классов**, которые содержат «оконные» компоненты GUI и определяют вид и поведение приложения. В мире Java библиотека классов называется Java™ Foundation Classes (JFC). Большая часть JFC состоит из набора GUI-компонентов по имени Swing. Набор обеспечивает GUI-компоненты для использования в приложениях и апплетах Java.

Набор компонентов Swing позволяет поставлять приложения со *сменным впечатлением и ощущением от использования*. Сменность имеет несколько значений. Она может означать соответствие с GUI-платформой, на которой выполняется программа (Windows, Unix и т. д.). Она может означать единую взаимную платформу впечатления и ощущения от использования, которая имеет одинаковое проявление на любой платформе выполнения. Swing вызывает это впечатление и ощущение от использования языка Java. Наконец, она

может означать настраиваемое программистом впечатление и ощущение от использования, уникальное для приложения.

Чтобы достичь **сменности**, большинство компонентов Swing сами являются платформо-независимыми или **легкими**, иногда называемыми бесподобными. Легкий компонент программируется без какого-либо использования платформо-зависимого (специального) кода. К сожалению, не все компоненты Swing легкие — некоторые являются **тяжеловесными**. В большинстве случаев, когда используются тяжеловесные компоненты, Swing обеспечивает обходные пути для скрытия специального кода, чтобы сменность впечатления и ощущения от использования была бы все еще достижима.

В Swing большинство конкретных классов, представляющих компоненты пользовательского интерфейса, называются, начиная с буквы J, например, `JDialog`, `JMenuBar`, `JButton`. Компоненты очень переплетены. Они могут быть составлены различными способами, чтобы обеспечить проектирование требуемого пользовательского интерфейса. Расклассифицировать компоненты пользовательского интерфейса в последовательные группы нелегко. Одной из грубых классификаций может быть следующая:

- контейнеры (например, `JInternalFrame` — класс «внутренняя структура» пакета Swing, `JTabbedPane` — класс «панель с закладками» пакета Swing);
- меню (например, `JPopupMenu` — класс «всплывающее меню» пакета Swing, `JRadioButtonMenuItem` — класс «элемент меню в виде переключателя» пакета Swing);
- элементы управления (например, `JRadioButton` — класс «переключатель» пакета Swing, `JScrollBar` — класс «полоса прокрутки» пакета Swing).

8.2.1. Контейнеры

Контейнеры — прямоугольные области на рабочем столе GUI, которые содержат другие компоненты, включая другие контейнеры, меню и элементы управления. В зависимости от предназначенной им цели и роли по отношению к другим контейнерам они называются окнами, диалоговыми окнами, частями окон, панелями и подобными терминами. Они определяют основные вид и поведение приложения.

Swing содержит разнообразные классы, которые могут сформировать контейнерные объекты. Четыре из этих классов — **тяжеловесные классы**: `JWindow` (класс «окно» пакета Swing), `JFrame` (класс «структура» пакета Swing), `JDialog` (класс «диалог» пакета Swing), и `JApplet` (класс «апплет» пакета Swing). В большинстве случаев основной контейнер программы — экземпляр тяжеловесного контейнера. Легкие контейнерные классы нуждаются в тяжеловесных компонентах для художественного оформления экрана и обработки событий. **Легкие классы** включают: `JInternalFrame` (класс «внутренняя структура» пакета Swing), `JDesktopPane` (класс «панель рабочего стола» пакета Swing), `JOptionPane` (класс «панель опций» пакета Swing), `JPanel` (класс «панель» пакета Swing), `JTabbedPane` (класс «панель с закладками» пакета Swing), `JScrollPane` (класс «прокручиваемая

панель» пакета Swing), JSplitPane (класс «расщепляющаяся панель» пакета Swing), JTextPane (класс «текстовая панель» пакета Swing), и JTable (класс «таблица» пакета Swing).

Часто приложение имеет только одно основное **первичное окно** и ряд **вторичных окон** (то есть окон, которые зависят от других окон; они нуждаются в родительском окне). JWindow — основной контейнер без «художественного оформления» — никаких границ, заголовка, строк меню и полос прокрутки. Из-за их отсутствия для реализации выпадающих окон обычно используются подклассы JWindow типа JFrame или JPanel. Объект JFrame имеет «художественное оформление». JPanel может использоваться, чтобы реализовать *диалоговое окно* — вторичное окно, предназначенное для того, чтобы показать сообщение и/или получать входную информацию от пользователя.

Рис. 16.1 является примером первичного и вторичного окон. Первичное окно по имени Figure 16-1 (рисунок 1) имеет тип JFrame. Вторичное окно по имени About (насчет) — типа JPanel.

Первичное окно на рис. 1 имеет строку меню (JMenuBar), панель инструментов (JToolBar) и кнопки в верхнем правом углу, чтобы сворачивать и закрывать окно. Элементы строки меню — экземпляры JMenuItem (класс «элемент меню» пакета Swing). Кнопки панели инструментов — экземпляры JButton (класс «кнопка» пакета Swing). У первичного окна можно изменять размеры с помощью любой стороны или угла его границы, а само окно можно перетащить (переместить) с помощью строки заголовка. Вторичное окно обычно имеет фиксированный размер.

JWindow, **JFrame** и **JPanel** — тяжеловесные компоненты. Чтобы иметь возможность добавлять легкие компоненты к тяжеловесному контейнерному объ екту, контейнер должен использовать специальный метод getContentPane() (получить содержимое панели), например:

```
aFrame.getContentPane().add(toolBar, BorderLayout.NORTH);
```

JDialog — другой тяжеловесный компонент для создания *диалоговых окон* (диалоговых устройств). Еще один компонент Swing — JOptionPane — обеспечивает ряд стандартных диалоговых окон. Все стандартные диалоговые окна являются *модальными*. Когда модальное окно активно (видимо в настоящий момент), пользователь не может использовать другие окна, пока не будет закончена обработка в модальном окне и окно не будет закрыто. Немодальное окно позволяет вводить в него информацию так же, как и в первичное окно, лишь поместив желаемое окно поверх остальных.

Приложение может запрашивать диалоговое окно, вызывая соответствующий метод в **JOptionPane**, например:

```
void helpAbout_ActionPerformed(ActionEvent e)
  {
    JOptionPane.showMessageDialog(this, new
      Frame1_AboutBoxPanel1(),
      "About", JOptionPane.PLAIN_MESSAGE);
  }
```

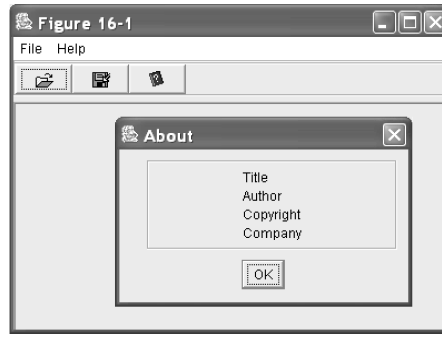


Рис. 1. Первичное и вторичное окна

JApplet — тяжеловесный компонент для создания **апплетов** — программ, которые выполняются внутри Web-страницы. Апплеты загружаются с сервера и могут быть кэшированы в Web-браузере для выполнения. `JWindow`, `JDialog` и `JApplet` зависят от `JFrame`. Следовательно, миними-зация `JFrame` минимизирует его окно, диалоговое окно или апплет. Удаление `JFrame` уничтожает также его окно, диалоговое окно или апплет и т. д.

Swing обеспечивает большое количество легких панелей, чтобы использовать их в других контейнерах. Одна из них — `JOptionPane`, которая реализует набор стандартных окон сообщений типа выдачи предупреждения, запроса подтверждения действия, объяснения события и т. д. Другие очень полезные панели включают `JTabbedPane`, `JScrollPane`, `JTextPane` и `JSplitPane`.

Объ ект **`JTabbedPane`** дает окно с рядом «страниц/панелей с закладками». **Страницы с закладками** накладываются друг на друга, так что в любой момент времени может быть видна только одна страница. Нажатие на закладку переводит ее страницу на передний план. Страницы с закладками очень полезны для работы с большим объ емом данных в одном диалоговом окне. Они занимают особое место в программировании апплетов, когда нельзя использовать выпадающие диалоговые окна.

Рис. 2 демонстрирует панель с закладками, содержащую две закладки с именами `Movie` (фильм) и `Actor` (актер). Добавление объекта `JPanel` к об-ъ екту `JTabbedPane` создает каждую закладку.

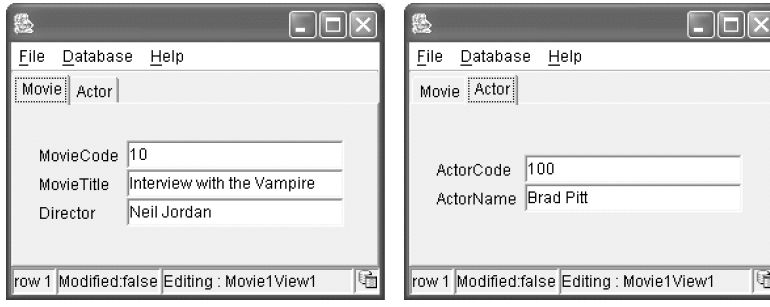


Рис. 2. Панель с закладками

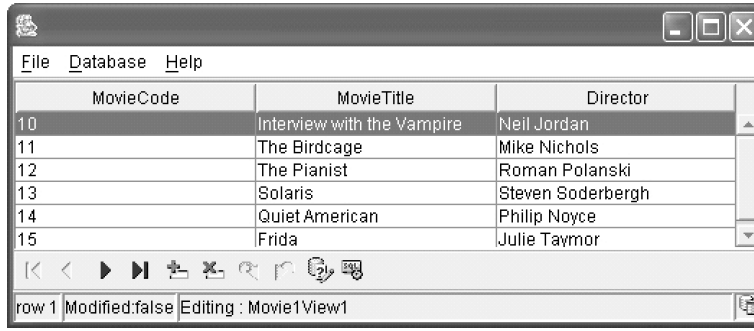


Рис. 3. Таблица

Объект **`JScrollPane`** обеспечивает возможность горизонтальной и вертикальной прокрутки окна. Можно выбрать различное сочетание **полос прокрутки**. Объект **JTextPane** отображает текст. Это — хороший кандидат, чтобы быть «прокручиваемым клиентом», то есть он может получить возможность прокрутки от `JScrollPane`. Расширяя (наследуя) `JEditorPane` (класс «панель редактирования» пакета Swing), `JTextPane` получает основные способности редактирования.

Объект **`JSplitPane`** обеспечивает *расщепляющееся окно* с двумя или большим количеством зависимых панелей, то есть действие на одной панели приводит к визуальным или другим изменениям в зависимой панели. Панели могут быть отделены вертикально или горизонтально. Windows Explorer является примером окна с «расщепляющейся панелью».

Один важный компонент Swing, который не поддается легкой классификации — **`JTable`**. Хотя он не совсем окно или панель, объект `JTable` является контейнером данных. `JTable` представляет таблицу строк и столбцов. Он может использоваться для всех видов сложных табличных манипуляций, включая функциональные возможности, наподобие электронных таблиц. Однако основное использование `JTable` в виде сетки — показать записи данных, полученные из реляционной БД.

Рис. 3 — пример окна, содержащего объект `JTable`. Средство про-крутки обеспечивается объектом `JScrollPane`. `JScrollPane` использует объект `JViewport` (класс «порт просмотра» пакета Swing), обеспечиваю-щий «порт просмотра» в источнике данных — содержании таблицы `Movie` БД, как показано в следующем фрагменте кода:

```
private JTable tableMovie1View1 = new JTable();
private JScrollPane scroller = new JScrollPane();
scroller.getViewport().add(tableMovie1View1, null);
```

Управление расположением

Swing использует **менеджер расположения**, чтобы разместить компоненты внутри контейнера. Если абсолютное расположение не запрограммиро-вано явно, компоненты динамически размещаются в пределах контейнера и корректируются в соответствии с измерениями контейнера. Размещение су-

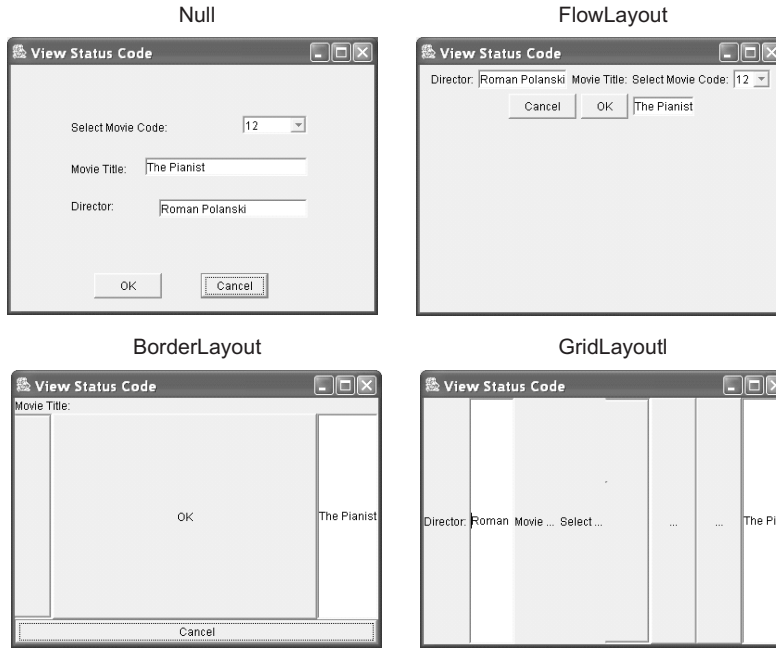


Рис. 4. Менеджеры расположения, примененные к одному и тому же набору компонентов

щественно отличается в зависимости от выбранного менеджера расположения. Метод `setLayout()` (установить расположение) позволяет выбрать желаемого менеджера, например:

```
aFrame.getContentPane().setLayout(new GridLayout(6,5));
```

Расположения, обеспечиваемые библиотекой Swing, включают `BorderLayout` (расположение границы), `FlowLayout` (расположение потока), `GridLayout` (расположение по сетке), `BoxLayout` (расположение в прямоугольнике) и `GridBagLayout` (расположение набора по сетке). **`BorderLayout`** — схема расположения, используемая по умолчанию для большинства контейнеров, кроме `JPanel` (который использует по умолчанию `FlowLayout`). Ручное расположение графических компонентов внутри контейнеров может быть запрограммировано с менеджером расположения, установленным в значение `null`:

```
aContainer.getContentPane().setLayout(null);
```

Объяснение детальных характеристик этих расположений вне возможностей данной книги. Рис. 4 показывает, как отличаются расположения, когда применяются различные менеджеры расположения к одному и тому же множеству компонентов. Расположение в верхнем левом углу — задаваемое вручную `null`-расположение. Остальные расположения были получены применением различных схем к исходному `null`-расположению.

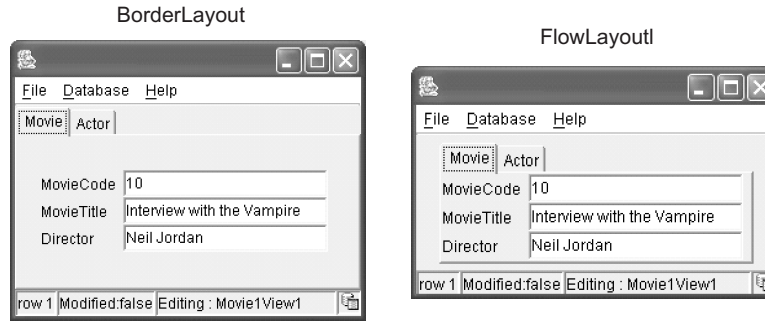


Рис. 5. Менеджеры расположения, примененные к панели с закладками

Рис. 4 может создать впечатление, что некоторые популярные расположения, типа `BorderLayout`, хуже других. На самом деле это не так. Выбор расположения должен быть таким, чтобы он соответствовал контейнеру. Рис. 5 показывает расположения `BorderLayout` и `FlowLayout`, приме-ненные к панели с закладками.

Управление выбором уровней

Приложение может открыть много окон, которые затем накладываются друг на друга. Класс Swing по имени **`JLayeredPane`** (класс «панель с уровнем» пакета Swing) добавляет глубину к контейнерам, поддерживая выбор уровней окон и других компонентов. Этот класс обеспечивает методы перемещения компонентов на передний план, на задний план или в соответствующее положение относительно друг друга.

Большинство компонентов помещается на стандартный (используемый по умолчанию) уровень. Компоненты **уровня, используемого по умолчанию**, соответственно накладываются друг на друга в зависимости от выбора пользователем этих компонентов (например, нажатие мышью окна делает его окном, расположенным поверх остальных). `JLayeredPane` позволяет также объ являть специальные характеристики уровней, которые заставят их вести себя предопределенным образом по отношению к другим уровням. Специальные случаи следующие:

- **уровень палитры**, который всплывает поверх уровня, используемого по умолчанию (например, всплывающая панель инструментов);
- **модальный уровень**, который помещается поверх всех других активных окон, панелей инструментов и палитр приложения и не допускает переключения к этим другим окнам, если он сам непосредственно не будет закрыт (например, модальное диалоговое окно);
- **выпадающий уровень**, который отображается временно в своем собственном уровне выше других уровней (например, комбинированная строка ввода, контекстное окно);
- **уровень перетаскивания**, который делает компонент видимым, когда он перетаскивается до того, как он будет опущен на уровень назначения.

Каждый уровень имеет отличительный целый номер (от 1 до 5). Уровень для компонента устанавливается, когда он добавляется к контейнеру. Текущее положение (глубина) компонента внутри уровня также обозначается целой величиной. Эта величина может быть определена непосредственно (`setLayer()` — установить уровень). Положение может быть изменено путем вызова `moveToFront()` (переместить на передний план) и `moveToBack()` (переместить на задний план).

8.2.2. Меню

Окна и некоторые другие компоненты могут содержать меню. Классы библиотеки Swing для поддержки меню включают **`JMenuBar`** (класс «строка меню» пакета Swing), **`JMenu`** (класс «меню» пакета Swing), **`JMenuItem`** (класс «элемент меню» пакета Swing), **`JCheckBoxMenuItem`** (класс «отмечаемый элемент меню» пакета Swing) и **`JRadioButtonMenuItem`** (класс «элемент меню в виде переключателя» пакета Swing). Модель классов, связанных с меню компонентов Swing, показана на рис. 16.6.

`JPopupMenu` (класс «всплывающее меню» пакета Swing), `JMenuBar`, `JMenu` и `JMenuItem` реализуют один и тот же интерфейс `MenuElement` (элемент меню). `JMenuItem` расширяет `AbstractButton` (абстрактная кнопка) — это проясняет, что `JMenuItem` является элементом управления . `JCheckBoxMenuItem` и `JRadioButtonMenuItem` — два вида `JMenuItem`.

Модель классов Swing для меню искажает факт, что объект `JMenuItem` содержится в объекте `JMenu`, а объект `JMenu` содержится в объекте `JMenuBar`. Никакое отношение включения (агрегирования) не используется в модели, чтобы представить этот факт. Возможно, представление, где `JMenu` является подклассом `JMenuItem`, — пример неподходящего использования наследования реализации.

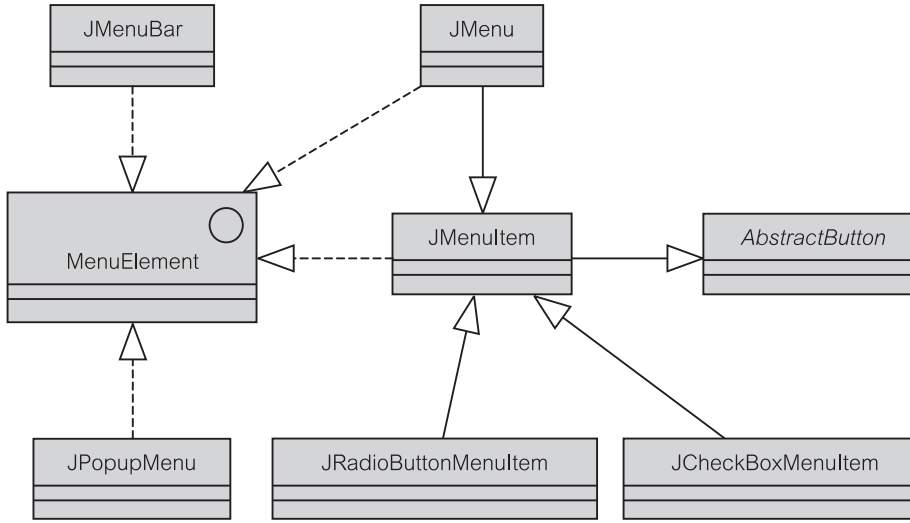


Рис. 6. Меню библиотеки Swing

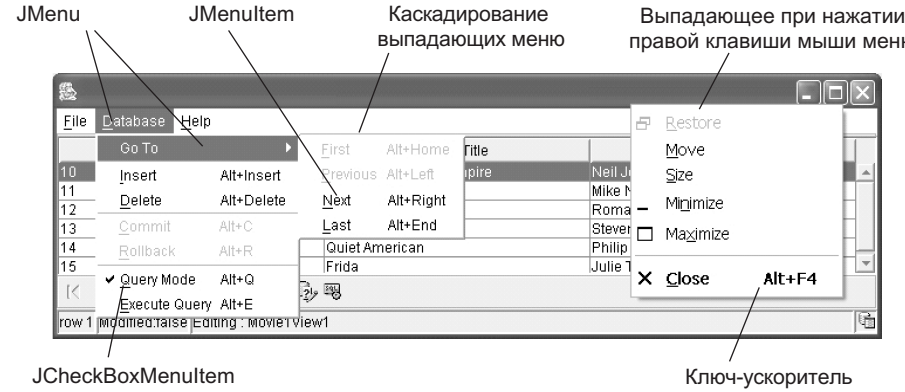


Рис. 7. Разные виды меню

Рис. 7 дает визуальное представление меню Swing. Строка меню (`JMenuBar`) содержит объекты `JMenu`. Элемент выпадающего объекта `JMenu` может быть объектом `JMenuItem` (например, `Insert` — добавить) или дру-гим объектом `JMenu` (например, элемент меню `Go To` — перейти к...). По-следний приводит к **каскадированию** («отображению справа») *всплывающего* меню (`JPopupMenu`). Другой пример `JPopupMenu` — **всплывающее при нажатии правой клавиши мыши** меню. Объект `JMenuItem` может быть пунктом `JCheckBoxMenuItem` (например, элемент `QueryMode` — режим за-проса) или выбором `JRadioButtonMenuItem`. Каждое меню может иметь ключ-ускоритель.

Рис. 8 объ ясняет использование панелей инструментов в проектирова-нии пользовательского интерфейса. Как правило, **панель инструментов** со-держит действия и элементы управления, которые дублируют функциональ-ные возможности нескольких наиболее полезных элементов меню. Это по-зволяет осуществлять более быструю активизацию данных элементов меню. Контейнер Swing по имени **`JToolBar`** (панель инструментов) помогает в проектировании панелей инструментов пользовательского интерфейса. Пане-ли инструментов могут иметь установленное местоположение в рамке окна или они могут быть плавающие, то есть, могут быть выделены из структуры в отдельное окно.

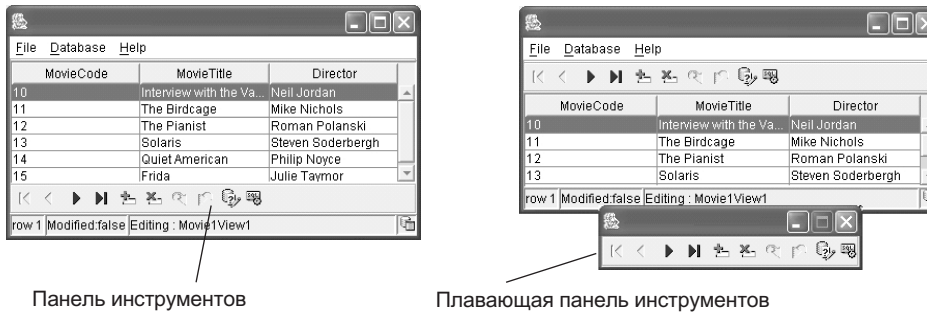


Рис. 8. Панели инструментов

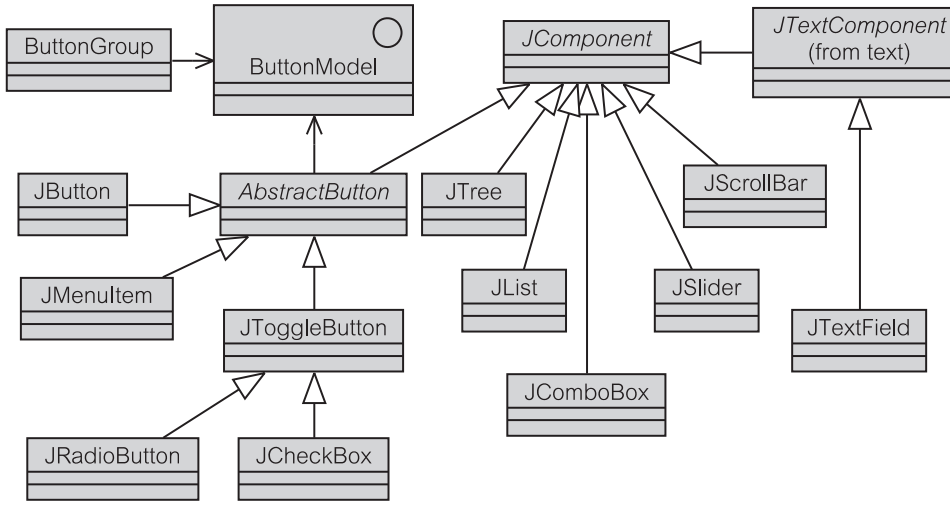


Рис. 16.9. Элементы управления Swing

8.2.3. Элементы управления

Элементы управления — сердце модели событий Swing (Swing event model). Они перехватывают, понимают и реализуют действия пользователя. Вообще говоря, они делятся на:

- командные *кнопки*, которые унаследованы от абстрактного класса по имени **`AbstractButton`** (абстрактная кнопка);
- *другие элементы управления*, которые унаследованы непосредственно от корневого абстрактного класса по имени `JComponent` (класс «компонент» пакета Swing).

Рис. 9 представляет модель классов для элементов управления Swing. Модель показывает, что имеются четыре категории кнопок: **`JButton`** (класс «кнопка» пакета Swing), **`JRadioButton`** (класс «кнопка-переключатель» пакета Swing), **`JCheckBox`** (класс «выключатель» пакета Swing), и **`JMenuItem`** (класс «элемент меню» пакета Swing), рассмотренная в предыдущем разделе. Чтобы реализовать находящееся во взаимосвязи поведение набора кнопок, они должны быть добавлены в компонент `ButtonGroup` (группа кнопок).

Рис. 10 дает визуальное представление кнопок и других элементов управления, перечисленных на рис. 9. Элементы управления реализованы с моделью событий Swing. Каждый элемент управления может иметь одного или большее количество объектов-подписчиков, которые «прислушиваются» к нему. Например, объектJTextField (класс «текстовое поле» пакета Swing) может публиковать события типа изменения текста, нажатия кнопки `Delete` (удалить) клавиатуры или выбора элемента Copy (копировать) меню.

`JButton` представляет собой обычную кнопку. В отличие от `JButton` кнопка `JToggleButton` (класс «кнопка-переключатель» пакета Swing) бу-

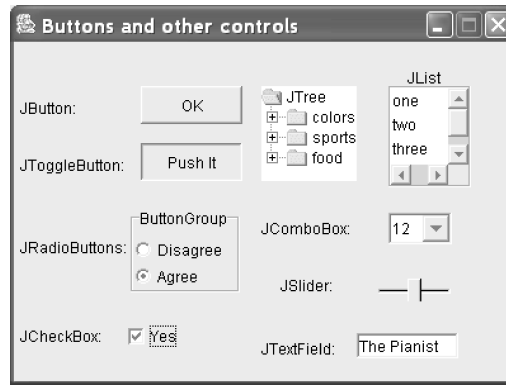


Рис. 10. Кнопки и другие элементы управления

дучи нажатой, остается нажатой (установленной в состояние true), пока она не нажата снова. Нажатая `JToggleButton` указывает, что кнопка активна. Две категории `JToggleButton` — это `JCheckBox` и `JRadioButton`.

`JCheckBox` изображается в виде квадрата и может быть установлен в true или false. Значение true **выключателя** обозначается «галочкой» на нем. `JRadioButton` используется, чтобы выбрать одну и только одну кнопку из множества кнопок, как, например, в случае выбора фильтра итерации 2 (рис. 14.3). Как и во многих механических устройствах, когда одна **кнопка-переключатель** «нажимается», все другие кнопки «отпускаются». Эти функциональные возможности получаются путем добавления каждой кнопки-переключателя в `ButtonGroup`.

`JComboBox` (класс «комбинированная строка ввода» пакета Swing) позволяет делать выбор из **выпадающего списка (комбинированная строка ввода)**. Комбинированная строка ввода часто называется на жаргоне **списком для выбора** (picklist). Объект `JComboBox` может быть редактируемым или нет (если нет, он не разрешает вводить собственные значения пользователя). По умолчанию объект `JComboBox` не редактируемый. `JComboBox` допускает выбор только одного значения.

`JList` (класс «список» пакета Swing) поддерживает выбор нескольких значений, используя типичные действия: `Ctrl-click` (Ctrl + нажатие клавиши мыши) и `Shift-click` (Shift + нажатие клавиши мыши). Выбор нескольких значений в `JList` возможен, потому что `JList` является окном со списком элементов (list box) фиксированного размера, хотя и прокручиваемым (обратите внимание, что прокрутка — не автоматическая особенность `JList`; она может быть получена включением `JList` в **`JScrollPane`** — класс «панель прокрутки» пакета Swing).

Значение и применимость компонентов **`JScrollBar`** (класс «линейка прокрутки» пакета Swing) и **`JSlider`** (класс «ползунок» пакета Swing) библиотеки Swing таковы, как это можно было бы ожидать. Полосы прокрутки могут быть добавлены к большинству контейнеров. Ползунки позволяют контролировать вход, наподобие регулятора громкости.

Последний, но не менее важный компонент, **`JTree`** (класс «дерево» пакета Swing) — очень полезный элемент управления и один из наиболее мощных в Swing [25]. В большинстве случаев простое представление дерева, показывающее иерархию различных элементов и контейнеров, как в Windows Explorer, является достаточным. Однако если приложение потребует более сложную реализацию дерева, в библиотеке Swing, вероятно, возникнет проблема.

Контрольные вопросы:

1. В чем состоят ключевые архитектурные различия между программируемым («толстым») клиентом и клиентом-браузером («тонким» клиентом)? Какие возможности локальной обработки данных имеет каждый из них?
2. Что включает в себя понятие «применимость» (usability) системы в рамках характеристик FURPS+?
3. Каковы требования к информационной обратной связи программы в случаях длительной обработки данных или фоновых системных операций?
4. Каким образом реализуется принцип «Снисходительность интерфейса» при ошибочных или недопустимых действиях пользователя?
5. Какие три группы факторов определяют необходимость адаптивности пользовательского интерфейса (квалификация пользователя, география/локализация, специальные возможности)?
6. Что означает понятие «сменное впечатление и ощущение от использования» (Pluggable Look and Feel) в Java Foundation Classes (JFC)?
7. Для какой цели в тяжеловесных контейнерах (например, JFrame) используется метод getContentPane() при добавлении графических элементов?
8. Какие задачи решают специализированные легковесные контейнеры JTabbedPane, JScrollPane, JSplitPane и JTable?
9. Какую функцию выполняют менеджеры расположения (Layout Managers) и какие стандартные менеджеры предоставляет библиотека Swing?
10. Каковы особенности менеджеров компоновки BorderLayout, FlowLayout и GridLayout, и как отменяется автоматическое размещение компонентов (null-расположение)?
11. В чем состоят функциональные различия между пунктами меню JCheckBoxMenuItem и JRadioButtonMenuItem?
12. От какого базового класса наследуются все командные кнопки и какую функцию выполняет класс ButtonGroup?